Project Assignment 4 Report: Medicaid

Task 1: Define and prepare the data

For the quality check, we used the same quality checks we used in sprint 3: we ensure the HCPCS_CODE follows a specific format and that the TOTAL_PAID is larger than 0. We added an AVERAGE_PAID column. This takes the TOTAL_PAID and TOTAL_CLAIMS and divides them to figure out the AVERAGE_PAID per claim for that month.

Task 2: Answer two analytical questions

For analysis 1, porting the DuckDB analysis from last spring to PySpark was relatively straightforward since they use extremely similar naming conventions. For analysis 2, we decided to derive the top billing provider for each HCPCS_CODE. This might be useful in determining if there's any odd billing providers skewing the data for our final sprint.

Task 3: Sample and prepare the final-sprint handoff

For the sample/source comparison, we compare the top paid HCPCS_CODEs. There is some overlap between the most common HCPCS_CODEs which is a great sign. We also decided to take path A for the analysis.

Task 4: Extend the benchmark

Through our benchmark we found that DuckDB still performed significantly faster than PySpark. We believe our PySpark implementation was probably not ideal, but it could also just be that PySpark is a much heavier tool then DuckDB.

Roles:

- **Team Members:**
    - Marius – Product Manager (Sprint 4)
    - Rupashi – Data Engineer (Sprint 4) author
    - Devin – Data Engineer (Sprint 4)
    - Dylan - Data Engineer (Sprint 4)
    - Justin – Data Storyteller (Sprint 4)

AI Disclosure:

AI was used for learning and debugging purposes only; code was hand-written with AI guidance, not AI generated. AI assistance was useful to understand new Spark commands.